

PotensicProxy_TAF

Projektdokumentation - Live Reco / Drone Reco

Version 0.956 | Android versionCode 29 | 01.10.2026

Ausgangsstand	0.955 Image Recognition Prerequisites
Ziel	Getrennte Camera- und Recognition-Pfade mit gemeinsamer Recognition-Pipeline
Status	Frontend typgeprueft; Hardwaretest und nativer Android-Compile noch erforderlich
Protokollregel	Bestaetigte Kamera-/Potensic-Protokollbereiche bleiben unveraendert

1. Ziel und Architektur

Die normale Foto-/Kamerafunktion bleibt vom Recognition-Weg getrennt. Fuer Recognition stehen zwei Bildquellen zur Verfuegung, die danach dieselbe Speicher-, Metadaten- und spaetere Erkennungspipeline verwenden.

- Live Reco: aktuelles dekodiertes LiveView-JPEG wird direkt auf Android gespeichert.
- Drone Reco: echtes Kamerafoto wird auf der Drohne erzeugt, nach Android uebertragen, lokal verifiziert und erst danach auf der Drohne geloescht.

Die eigentliche KI-/Objekterkennung ist in Version 0.956 noch nicht implementiert. Der Datenaufbau enthaelt jedoch bereits einen vorbereiteten Recognition-Ergebnisbereich.

LiveView frame ----|

> Recognition image -> Metadata -> Recognition engine -> Results

Camera photo -----/

2. Physische Speichertrennung

Normale Kamera-Downloads:

Pictures/PotensicProxy/Camera/

Recognition-Aufnahmen:

Pictures/PotensicProxy/Recognition/

Beide Pfade werden ueber Android MediaStore veroeffentlicht. Zusaetzlich fuehrt die App einen eigenen Index mit content://-URI, Quelle, Hash, Verifikationsstatus und optionalem Recognition-Metadatensatz.

3. Gallery-Struktur

Die Gallery-Seite besitzt zwei getrennte Tabs:

- Camera - bestehende Kamera-/SD-Galerie und normaler Download.
- Recognition - lokale Recognition-Bilder aus beiden Aufnahmequellen.

Recognition kann nach All, Live und Drone gefiltert werden. Jedes Bild zeigt seine Quelle, den Verifikationsstatus und auf Wunsch den vollstaendigen Metadatenatz.

4. Live Reco

1. Metadaten zum Aufnahmezeitpunkt erfassen.
2. Letztes dekodiertes LiveView-JPEG uebernehmen.
3. Unter Pictures/PotensicProxy/Recognition/ speichern.
4. MediaStore-Datei zuruecklesen.
5. Byteanzahl und SHA-256 mit dem Quelldatenatz vergleichen.
6. Bei erfolgreicher Verifikation als Recognition-Eingabe bereitstellen.

Vorteil: sehr geringe Latenz und kein temporaeres Standbild auf der Drohne. Nachteil: Bildqualitaet und Detailgrad sind durch den LiveView-Stream begrenzt.

5. Drone Reco

1. Aktuelle Kamera-Dateiliste vollstaendig einlesen.
2. Kamera-Galerie verlassen.
3. Recognition-Metadaten unmittelbar vor dem Ausloesen erfassen.
4. Normalen Potensic-Fotobefehl senden.
5. Kamera-Galerie erneut oeffnen und neue Datei gegen die Ausgangsliste bestimmen.
6. Originalfoto nach Android uebertragen.
7. Unter Pictures/PotensicProxy/Recognition/ speichern.
8. Gespeicherte MediaStore-Datei zuruecklesen.
9. Byteanzahl und SHA-256 verifizieren.
10. Nur bei verified=true den Loeschbefehl fuer exakt die Quelldatei senden.
11. Kamera-Loeschbestaetigung abwarten.
12. Vorgang erst danach als erfolgreich abschliessen.

Sicherheitsregel: Bei fehlendem neuen Foto, Downloadfehler, Speicherfehler, Verifikationsfehler oder fehlender Loeschbestaetigung wird kein erfolgreicher Auto-Delete-Abschluss gemeldet. Die Quelldatei soll in diesen Fehlerfaellen auf der Drohne verbleiben.

6. Recognition-Metadaten

Schema-Version 1 speichert maximal die zum Aufnahmezeitpunkt tatsaechlich verfuegbaren Daten. Nicht bekannte Werte werden nicht geschaetzt.

Gruppe	Gespeicherte Daten
Zeit/Quelle	Unix-Zeit, ISO-Zeit, LIVE_RECO oder DRONE_RECO
Position/Flug	Latitude, Longitude, GPS-Gueltigkeit, relative Hoehe, Altitude, Distanzen, Geschwindigkeiten, Flugstatus
Lage	Heading, Pitch, Roll
Gimbal	Pitch, Roll, Yaw, Geschwindigkeiten, Steuerwert, Status
Steuerung	RC-Hardwarewerte, virtuelle/angeforderte Werte, Telemetrie-RC-Werte
Kamera	Foto-/Video-Auflösungsindex, EV, Zoom, Manual, Shutter, ISO, WB, RAW, OSD, Photo-GPS, SD-Status
System	Verbindungsstatus, App-Version, Metadaten-Schema
Bild	Format, Bytegrösse, Bildabmessungen soweit dekodierbar, SHA-256, MediaStore verified
Recognition	processed=false und runs=[] als spaeterer Ergebnisbereich

7. Gemeinsame Pipeline und spaeterer Hybrid-Modus

Live Reco und Drone Reco erzeugen denselben AndroidStoredImage-/Recognition-Datensatz. Das spaetere Modell muss daher nicht fuer zwei Aufnahmewege doppelt implementiert werden.

Ein Hybrid-Modus kann spaeter Live Reco zur schnellen Kandidatensuche verwenden und bei passenden Treffern automatisch Drone Reco fuer eine hochaufgeloeste Bestaetigung ausloesen.

8. Geaenderte Kernmodule

- app/src/main/java/com/potensic/proxy/AndroidMediaRepository.kt
- app/src/main/java/com/potensic/proxy/WebServer.kt
- webui/src/services/AndroidMediaService.ts
- webui/src/services/CameraMediaService.ts
- webui/src/services/RecognitionCaptureService.ts (neu)
- webui/src/services/RecognitionMetadataService.ts (neu)
- webui/src/services/DroneControlService.ts
- webui/src/components/gallery/GalleryView.vue
- webui/src/i18n/index.ts
- README.md
- app/build.gradle.kts / webui/package*.json

9. Verifikation

Erfolgreich: Vue-/TypeScript-Typpruefung.

```
./node_modules/.bin/vue-tsc --noEmit
```

PotensicProxy_TAF 0.956 - Live Reco / Drone Reco

Ergebnis: keine TypeScript-/Vue-Typfehler.

Der vollstaendige Vite-Build wurde versucht:

```
npm run build
```

Ergebnis: Abbruch wegen fehlendem optionalen Linux-Rollup-Paket @rollup/rollup-linux-x64-gnu im uebernommenen node_modules. Die vorhandenen node_modules enthalten Windows-Rollup-Binaries. Fuer den Windows-Build-PC ist ein sauberes npm ci vor dem Build vorgesehen.

Der Android-Kotlin-Compile wurde ebenfalls angestossen:

```
./gradlew :app:compileDebugKotlin --offline
```

Ergebnis: Gradle 8.11.1 ist in der Arbeitsumgebung nicht lokal vorhanden; der Wrapper kann services.gradle.org wegen gesperrtem Netzwerk nicht erreichen. Daher kein vollstaendiger nativer Compile in dieser Umgebung.

10. Hardwaretest / Freigabekriterien

- Live Reco erzeugt sofort ein Bild unter Pictures/PotensicProxy/Recognition/.
- Drone Reco erkennt nach dem Ausloesen genau die neu entstandene Datei.
- Das Originalfoto wird vollstaendig uebertragen.
- MediaStore-Verifikation liefert verified=true.
- Der Loeschbefehl entfernt exakt die verifizierte Quelldatei.
- Die Kamera-Loeschbestaetigung wird zuverlaessig empfangen.
- Bei absichtlich unterbrochenem Transfer wird die Quelldatei nicht geloescht.
- Camera-Downloads bleiben unveraendert und werden nicht automatisch geloescht.

Bis zu diesem Test ist die Auto-Delete-Logik implementiert, aber nicht hardwareseitig bestaetigt.

11. Nicht Bestandteil von 0.956

- ML-/Objekterkennungsmodell selbst.
- Automatische Hybrid-Reco-Triggerlogik.
- Zusaetzliche EXIF-/XMP-Einbettung in die Bilddatei. Der vollstaendige TAF-Metadatenatz liegt derzeit im app-eigenen Index und ist in der Recognition-Gallery sichtbar.